VSMR ↔ Supersmart ESP-NOW Link (Supersmart side)

Branch: `VSMR-integration`

Purpose

Supersmart receives 3-phase voltages from a **VSMR** unit over **ESP-NOW** while staying connected to Wi-Fi. The link is enabled from the techsetup page and secured by a **manually-entered VSMR MAC filter**.

New files

- **`src/vsmr_link_proto.h`** — shared wire protocol (byte-identical copy exists in the VSMR project). Minimal by design: 4-byte version/sync word `"VS-1"`, **no magic / CRC / ACK / sequence number**. Two frame types:
  - `VLINK_BEACON` — Supersmart → broadcast: advertises current channel + its STA MAC (so a VSMR can discover it).
  - `VLINK_DATA` — VSMR → Supersmart: 3 phase voltages in **deci-volts** (×10, e.g. 2301 = 230.1 V) + status flags + relay state.
  - Includes a core-2.x/3.x receive-callback signature shim.
- **`src/vsmr_link.h` / `src/vsmr_link.cpp`** — the receiver module. Public API:
  - `void initVsmrLink(bool enabled, const char *peerMac)` — call once in `setup()` after Wi-Fi is up.
  - `void serviceVsmrLink()` — non-blocking housekeeping; call from `core0Task`.
  - `void setVsmrLinkEnabled(bool)` / `bool vsmrLinkEnabled()` — runtime enable (persistence is handled externally, see below).
  - `void vsmrLinkSetPeerMac(const char*)` / `String vsmrLinkPeerMacStr()` — MAC filter set/get.
  - `bool vsmrLinkAlive()` — true if a valid frame arrived within the window.
  - `bool vsmrLinkGetVoltages(float &r, float &y, float &b, uint8_t &flags, bool &relay)` — **read the received voltages here** (returns true if fresh).

Module behavior

- On enable: `WiFi.setSleep(false)` (ESP-NOW frames are dropped during Wi-Fi modem sleep), `esp_now_init`, registers a broadcast peer.
- **Beacon** (channel + own STA MAC) broadcast ~1 Hz **only during a manual pairing window** (Button 1 held 5 s; see below). Supersmart does NOT auto-beacon on link loss — a paired VSMR reconnects on its own via PROBE (send to the known MAC), so outside a pairing window Supersmart is silent.
- **Receive path:** validates `"VS-1"`, checks it is a DATA frame, then applies the **MAC filter** — accepts only if `src_addr == configured VSMR MAC` (blank config = accept any). Stores latest voltages + `rxLastMs`. **No ACK is sent** (a dropped sample is superseded by the next).

Button-1 pairing broadcast

Hold **Button 1 (SWITCH_1) alone for 5 s** (all other buttons released) to broadcast the beacon for a short pairing window. This lets a VSMR that was just put into learning mode (its own 5 s button) rediscover this Supersmart.

- `checkVsmrPairing()` in `power_monitoring.cpp` (called from the main loop, same pattern as `checkFactoryReset` / `checkDisableWiFi`) detects the 5 s hold, calls

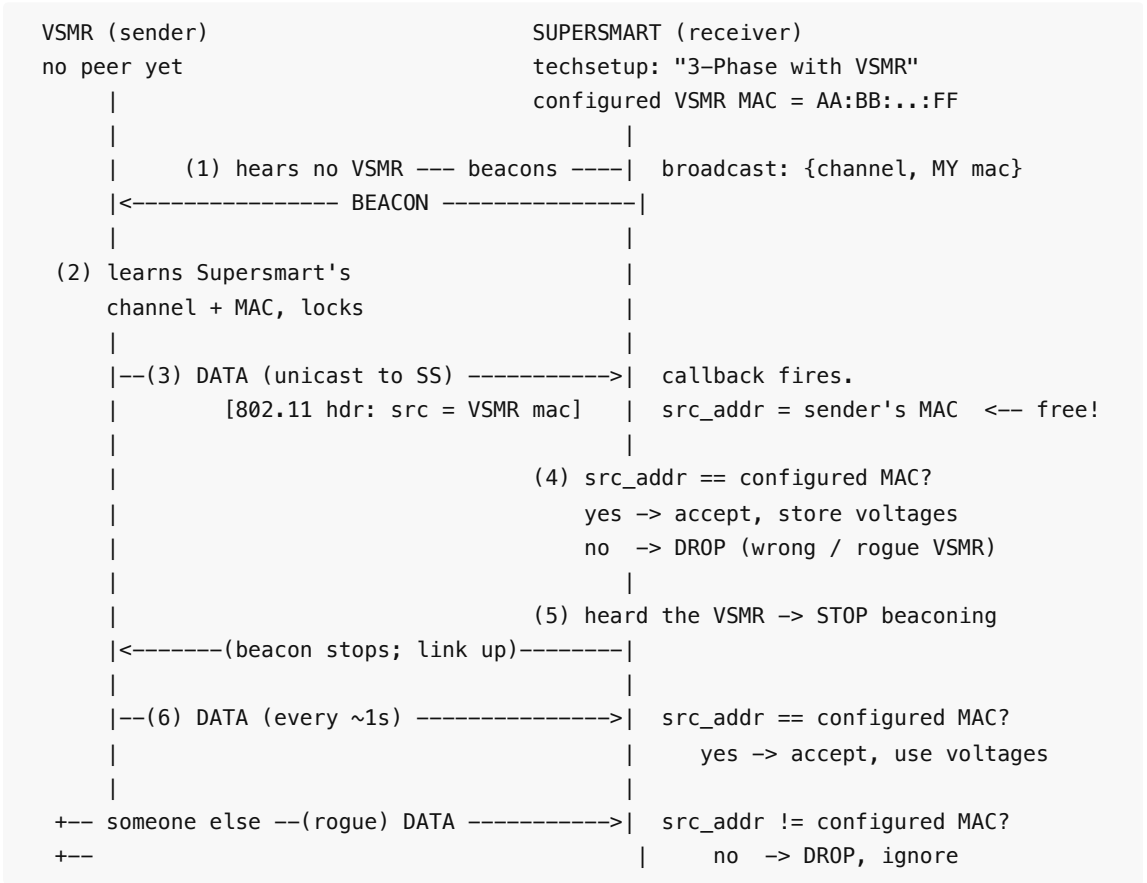
`vsmrLinkStartPairing()`, and lights `vsmrPairLedIndicate` (LED_1 steady ~2 s) as feedback. It also sets `ignoreButton1FallOnce` so Button 1's normal short-press action does not fire from the same hold.

- `vsmrLinkStartPairing()` loads a beacon **countdown**: `beaconsLeft = PAIRING_BEACONS` (120 beacons at ~1 Hz ≈ 2 min). No deadline math — the counter is the whole state.
- `serviceVsmrLink()` sends one beacon/second while `beaconsLeft > 0` (decrementing each). The window **closes early** the instant a VSMR frame is heard (`vsmrLinkAlive()` → `beaconsLeft = 0`), so a successful pair stops beaconing immediately; otherwise it stops after the 120 beacons. It is a true one-shot, so a later brief VSMR drop never re-opens beaconing.

Full re-pair procedure: set the new VSMR's MAC in techsetup → hold the VSMR's button 5 s (it forgets + LISTENs) → hold this Supersmart's Button 1 5 s (it beacons up to ~2 min, or until the VSMR is heard) → VSMR learns this Supersmart.

How the MAC filter works

Key fact: every received ESP-NOW frame already carries the **sender's MAC** — it is in the 802.11 header and delivered to the receive callback for free (`esp_now_recv_info_t->src_addr` on core 3.x). Supersmart does not need any handshake or "pairing" packet to know who sent a frame. With **manual MAC entry**, Supersmart simply compares that `src_addr` to the MAC you typed in techsetup and accepts the frame only if they match.



Notes:

- If the configured MAC is **blank**, the filter is OFF (step 4/6 accept any sender) — useful for bring-up before you know the VSMR's MAC.

- Supersmart never *learns* the MAC; it only ever compares against what you typed. To re-pair after a VSMR board swap, edit the MAC in techsetup.
- A rogue that spoofs the configured MAC would still pass (MAC filtering only); see "Current limitations".

Modified files

- **`src/preference_management.h` / `.cpp`**
  - `save3PhaseSettings(...)` and `read3PhaseSettings(...)` each gained **two trailing params**: `bool vsmrLinkEnabled` and `String vsmrMac`.
  - Persisted in the **`"Admin-config"`** NVS namespace, keys **`vsmrLink`** (bool) and **`vsmrMac`** (string).
- **`src/main.cpp`**
  - `read3PhaseSettings(...)` call now also reads `vsmrLinkConfig` + `vsmrMacConfig`.
  - `initVsmrLink(vsmrLinkConfig, vsmrMacConfig.c_str())` in `setup()`.
  - `serviceVsmrLink()` called inside `core0Task`.
- **`src/api.cpp`** — `/adminapi/3phasesettings`:
  - `"3p"` is now **tri-state**: `0` = not 3-phase, `1` = 3-phase, `2` = 3-phase **with VSMR**.
  - Parses `vsmrMac`; calls `save3PhaseSettings(..., vsmrLinkOn, vsmrMac)`, `setVsmrLinkEnabled(vsmrLinkOn)`, `vsmrLinkSetPeerMac(vsmrMac)`.
  - Settings GET returns `"3p"` (2 when enabled) and `"vsmrMac"`.
- **`src/mqtt.cpp`** — MQTT admin-settings path mirrors api.cpp (same tri-state parse, save, runtime apply, and `vsmrMac` in the report).
- **`data/techsetup.html`** — option **C "3-Phase with VSMR"** (value `2`) added to the 3-phase radio, plus a **"Paired VSMR MAC address"** text input (`#vsmr_mac`).
- **`data/scripts/adminscript.js`** — sends `vsmrMac` in the settings payload (with `AA:BB:CC:DD:EE:FF` validation) and populates the field on load. Note: `web_content.h` / `web_mappings.h` regenerate from `data/` at build via the pre-script.

User flow

Techsetup → Q3 → **C. 3-Phase with VSMR** → enter the VSMR's MAC (read it from the VSMR's own web portal "Device MAC address"). Enable flag + MAC persist in `Admin-config`.

Current limitations / next steps

- **Received voltages are stored but NOT yet consumed** — nothing feeds `vsmrLinkGetVoltages()` into Supersmart's power-monitoring / MQTT / decision logic. This is the obvious next integration point.
- MAC filter is **one-directional** (Supersmart trusts the VSMR's MAC; VSMR does not authenticate Supersmart's beacon).
- **No encryption / HMAC** — MAC filtering only.
- Blank MAC = filter off (accepts any VSMR).
- Reconnection relies on the VSMR side (unicast MAC-layer send callback); Supersmart just re-beacons after 10 s of silence.

Build status

`envS3C1` builds successfully (Flash ~73.2%).